

Snake Game Class Overview

Mahum Moazzam

Project split into two layers:

- **Core logic layer:** plain C# classes, no Unity dependencies. Contains all game rules to be tested.
- **Presentation layer:** thin `MonoBehaviour` scripts to read core logic's state, render `GameObjects`, read player input.

Making game logic testable without needing to run Unity's play mode.

Core Logic Layer (plain C#)

`GridPos` (struct)

Single cell coordinate on game board (`x`, `y`). Immutable value type. Used everywhere a position needs to be compared or moved.

`Direction` (enum)

`Up`, `Down`, `Left`, `Right` + small helper to check oppositional directions and block snake reversing into itself.

`SnakeBody`

Owens the snake's list of `GridPos` segments and its current `Direction`. Responsibilities:

- `Move()`: moves the head by one cell in the current direction + takes rest of the body along
- `Grow()`: adds 1 segment instead of shrinking the tail on the next move
- `ChangeDirection(Direction)`: updates direction, ignoring illegal 180° reversals
- `CollidesWithSelf()`: checks whether the head overlaps any other body segment

`FoodSpawn`

- `SpawnNewPosition(GridSize, occupiedCells)`: returns random `GridPos` that isn't currently occupied by the snake

GameBoard

Holds board dimensions and maintains boundaries.

- `IsWithinBounds(GridPos)`: wall-collision checks

GameState (enum)

`Playing`, `Paused`, `GameOver`. Drives **State pattern**. `SnakeGame` class below behaves differently depending on which state it's in.

SnakeGame

Core rules engine. Composes `SnakeBody`, `FoodSpawn`, `GameBoard`, and tracks `Score` and `GameState`. Responsibilities:

- `Tick()`: moves the snake, checks food/self/wall collisions, updates score, updates state
- Raises events (`OnScoreChanged`, `OnGameOver`) rather than reaching out to Unity UI directly. Observer pattern, detaching core logic from presentation

Zero Unity dependencies, so it's the primary target of unit tests.

Presentation Layer (Unity MonoBehaviour)

GameManager (MonoBehaviour, Singleton)

Owns the single `SnakeGame` instance for the scene, drives its `Tick()` on a fixed timer, and is the single access point other scripts use to reach game state. Singleton pattern used because one game running per scene and multiple scripts need a shared reference to it.

InputController (MonoBehaviour)

Reads keyboard input each frame and calls `SnakeGame.ChangeDirection()` through `GameManager`. Contains no game rules itself.

SnakeRend (MonoBehaviour)

Subscribes to the core `SnakeGame` and, on each `Tick()`, instantiates/moves segment `GameObjects` to match `SnakeBody`'s list of `GridPos`. Converts grid coordinates to world-space `Vector3` positions.

FoodRend (MonoBehaviour)

Moves the food GameObject to match the core logic's current food `GridPos` whenever a new one is spawned.

UIManager (MonoBehaviour)

Subscribes to `GameManager`'s `OnScoreChanged` and `OnGameOver` events and updates the score text / game-over panel accordingly. Never polls state directly; reacts to events.

Design Pattern Summary

Pattern	Where	Why
Singleton	<code>GameManager</code>	One authoritative game instance per scene, accessed from multiple unrelated scripts
Observer	<code>SnakeGame</code> events → <code>UIManager</code> , renderers	Decouples core rules from Unity presentation code; core logic never references UI or GameObjects
State	<code>GameState</code> enum driving <code>SnakeGame.Tick()</code> behaviour	Cleanly separates "what happens while playing" from "what happens once game over" without nested conditionals scattered everywhere

Planned Unit Test Targets

All against `SnakeGame` / `SnakeBody` directly, no Unity runtime required:

1. Snake moves one cell in the current direction on `Tick()`
2. Snake grows (adds a segment) after eating food
3. Game transitions to `GameOver` when the snake collides with itself
4. Game transitions to `GameOver` when the snake hits the board boundary